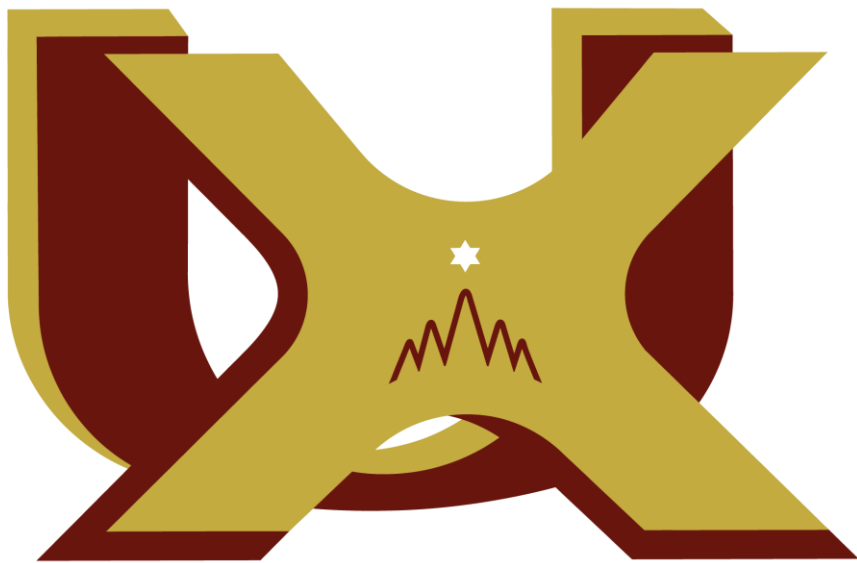


FACTORIAL REACONDICIONADO

Héctor Luis García Marthen

Universidad Xalapa



UNIVERSIDAD DE XALAPA

• ————— •
Saber / Trascender

PREGUNTAS RELACIONADAS AL TEMA:

- 1. ¿Qué es la recursividad?** Es una técnica de programación en la que una función se llama a sí misma dentro de su propia definición para resolver un problema dividiéndolo en subproblemas más pequeños del mismo tipo.
- 2. ¿Cuál es el caso base de la función factorial()?** Es la condición que detiene las llamadas recursivas
- 3. ¿Cuál es el caso recursivo?** Es la sección donde la función se llama a sí misma con un subproblema reducido

4. ¿Qué sucede si eliminamos el caso base?

La función entraría en una recursión infinita, llamándose a sí misma indefinidamente ($n - 1, n - 2, n - 3, \dots$). Esto agota la memoria reservada para las llamadas a funciones y provoca un fallo por desbordamiento de pila (**Stack Overflow**).

5. ¿Qué ocurre con el valor de n en cada llamada?

El valor de n se decrementa en 1 en cada llamada ($\text{factorial}(n - 1)$), acercando paulatinamente el argumento hacia el caso base (0).

6. ¿Qué relación existe entre la recursividad y la pila de llamadas (call stack)?

Cada vez que se realiza una llamada recursiva, se crea un nuevo marco de pila (*stack frame*) que almacena las variables locales y el punto de retorno de esa ejecución en la memoria del sistema. Estos marcos se van apilando ("fase de apilado") y no se liberan hasta que el caso base responde y las llamadas empiezan a retornar en orden inverso ("fase de retorno" / LIFO: *Last In, First Out*).

7. ¿Cuál de las dos implementaciones consume más memoria?

La implementación **recursiva** consume significativamente más memoria. La versión iterativa (usando un bucle `for` o `while`) utiliza una cantidad constante de memoria $O(1)$, mientras que la recursiva requiere espacio proporcional a n , ocupando memoria en la pila $O(n)$ por cada marco de función creado.

8. ¿Cuál implementación consideras más sencilla de comprender? ¿Por qué?

La **recursiva** suele ser conceptualmente más intuitiva para problemas matemáticos como el factorial, ya que refleja directamente la definición matemática del problema: $n! = n \times (n - 1)!$. Sin embargo, la versión **iterativa** suele ser más fácil de rastrear mentalmente en términos de ejecución paso a paso en memoria sin requerir visualizar la pila.

9. ¿En qué situaciones podría ser conveniente utilizar una solución recursiva?

- **Estructuras de datos jerárquicas o recursivas:** Recorrido de árboles (árboles binarios, DOM) y grafos.
- **Algoritmos de "Divide y Vencerás":** Tales como *QuickSort*, *MergeSort* o búsqueda binaria.

- **Exploración de espacios de solución:** Algoritmos con *backtracking* (resolver laberintos, sudoku, N-reinas).

-

10. ¿Qué limitación puede presentar el cálculo recursivo del factorial para números muy grandes?

Presenta dos limitaciones principales:

1. **Desbordamiento de pila (*Stack Overflow*):** Un n extremadamente grande creará demasiados marcos de función en la pila antes de llegar al caso base, agotando la memoria de la pila.
2. **Desbordamiento numérico (*Integer Overflow*):** El factorial crece a un ritmo extremadamente rápido. Incluso usando long long (64 bits), el valor máximo representable se desborda rápidamente a partir de $21!$.

FUENTES DE INFORMACIÓN

Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1988). *Estructuras de datos y algoritmos*. Addison-Wesley Iberoamericana.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3.^a ed.). MIT Press.

Joyanes Aguilar, L. (2008). *Fundamentos de programación: Algoritmos, estructura de datos y objetos* (4.^a ed.). McGraw-Hill.

Prata, S. (2012). *C++ Primer Plus* (6.^a ed.). Addison-Wesley Professional.

Stroustrup, B. (2013). *The C++ programming language* (4.^a ed.). Addison-Wesley Professional.